Redis vs. Memcached: In-Memory Data Storage Systems

Alibaba Cloud Follow
May 18 · 15 min read

Redis and Memcached are both in-memory data storage systems. Memcached is a high-performance distributed memory cache service, and Redis is an open-source key-value store. Similar to Memcached, Redis stores most of the data in the memory. It supports operations on various data types including strings, hash tables, and linked lists among others. The Alibaba Cloud ApsaraDB family supports these two popular data storage systems: ApsaraDB for Redis and ApsaraDB for Memcache. In this article, we will examine the difference between Redis and Memcached.

Feature Comparison

Salvatore Sanfilippo, the author of Redis, shared the following points of comparison between Redis and Memcached:

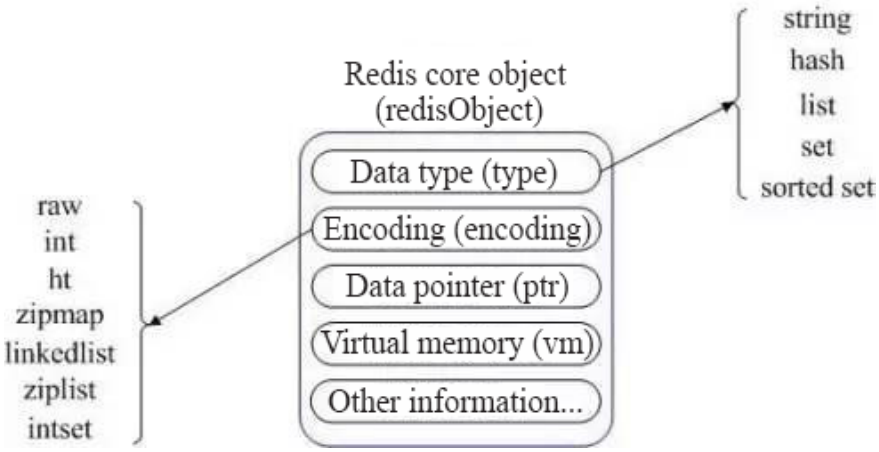
- Server-end data operations
- Redis supports server-end data operations, and owns more data structures and supports richer data operations than Memcached. In Memcached, you usually need to copy the data to the client end for similar changes and then set the data back. The result is that this greatly increases network IO counts and data sizes. In Redis, these complicated operations are as efficient as the general GET/SET operations. Therefore, if you need the cache to support more complicated structures and operations, Redis is a good choice.
- Memory use efficiency comparison
- Memcached has a higher memory utilization rate for simple key-value storage. But if Redis adopts the hash structure, it will have a higher memory utilization rate than Memcached thanks to its combined compression mode.
- Performance comparison
- Redis only uses single cores while Memcached utilizes multiple cores. So on average, Redis boasts a higher performance than

Memcached in small data storage when measured in terms of cores. Memcached outperforms Redis for storing data of 100k or above. Although Redis has also made some optimizations for storing big data, it is still inferior to Memcached.

Let us now discuss some points to support the above comparisons.

Different Data Types Supported

Unlike Memcached which only supports data records of the simple key-value structure, Redis supports much richer data types, including String, Hash, List, Set and Sorted Set. Redis uses a redisObject internally to represent all the keys and values. The primary information of the redisObject is as shown below:



The type represents the data type of a value object. The encoding indicates the storage method of different data types in the Redis, such as type=string represents that the value stores a general string, and the corresponding encoding may be raw or int. If it is int, Redis stores and represents the associated string as a value type. Of course, the premise is that it is possible to represent the string by a value, such as strings of "123" and "456". Only upon enabling the Redis virtual memory feature will it allocate the vm fields with memory. This feature is off by default. Now let us discuss some data types.

String

Frequently used commands: set/get/decr/incr/mget and so on.

Application scenarios: String is the most common data type and general key/value belong to this category.

Implementation method: String is a character string by default in Redis referenced by redisObject. When called for the INCR or DECR operations, the system will convert it to the value type for computation. At this time, the redisObject's encoding field is int.

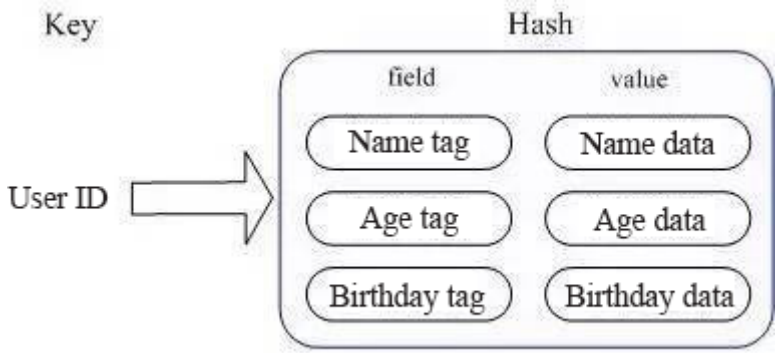
Hash

Frequently used commands: hget/hset/hgetall and so on.

Application scenarios: Storing user information object data, including the user ID, user name, age and birthday; retrieving the user name, age or birthday through the user ID.

Implementation method: Hash in Redis is a HashMap of the internally stored value and provides the interface for direct access to this map member. As shown in the figure, the Key is the user ID and the Value is a map. The key of this map is the member attribute name, and the value is the attribute value. In this way, one can directly execute changes and have access to the data through the internal map's key (In Redis, the internal map key is called field), that is, through the key (user ID) + field (attribute tag) to perform operations on the corresponding attribute data.

There are two ways of implementation for the current HashMap: when there are only a few members in the HashMap, Redis opts for one-dimensional arrays for compact storage to save memory, instead of the HashMap structure in the real sense. At this time, the encoding of the corresponding value redisObject is zipmap. When the number of members increases, Redis will convert them into the HashMap in the real sense and the encoding at this time will be ht.



List

Frequently used commands: lpush/rpush/lpop/rpop/lrange.

Application scenarios: The Redis List is the most important data structure in Redis. In fact, it is possible to implement Twitter's following list and fans list using the list structure of Redis.

Implementation method: Through a two-way linked list which supports reverse lookups and traversing to facilitate operations. But it also brings about some additional memory overhead. Many implementations in Redis, including sending buffering queues, also adopt this data structure.

Set

Frequently used commands: sadd/spop/smembers/sunion and so on.

Application scenarios: Redis set provides a similar external list function as list does. It is special in that the set can automatically remove duplicates. When you need to store a list of data without any duplication, the set is a good option. In addition, the set provides an important interface to judge whether a member is within a set, a feature not provided by list.

Implementation method: The internal implementation of set is a HashMap whose value is always null. It actually removes duplicates quickly through calculating the hash values. In fact, this is also the reason why set can judge whether a member is within the set.

Sorted Set

Frequently used commands: zadd/zrange/zrem/zcard and so on.

Application scenarios: The application scenarios of Redis sorted set are similar to those for the set. The difference is that while set does not automatically sort the data, sorted set can sort the members through a priority parameter (score) provided by the user. What's more, the latter also sorts inserted data automatically. You can choose the sorted set data structure when you need an orderly set list without duplicate data, such as Twitter's public timeline, which can take the posting time as the score for storage with automatic sorting of the data obtained by time.

Implementation method: Redis sorted set uses HashMap and SkipList internally to ensure efficient data storage and order. HashMap stores the mappings between the member and the score; while the SkipList stores all the members. The sorting relies on the score stored in the HashMap. Using the SkipList structure can improve the search efficiency and simplify the implementation.

Different memory management scheme

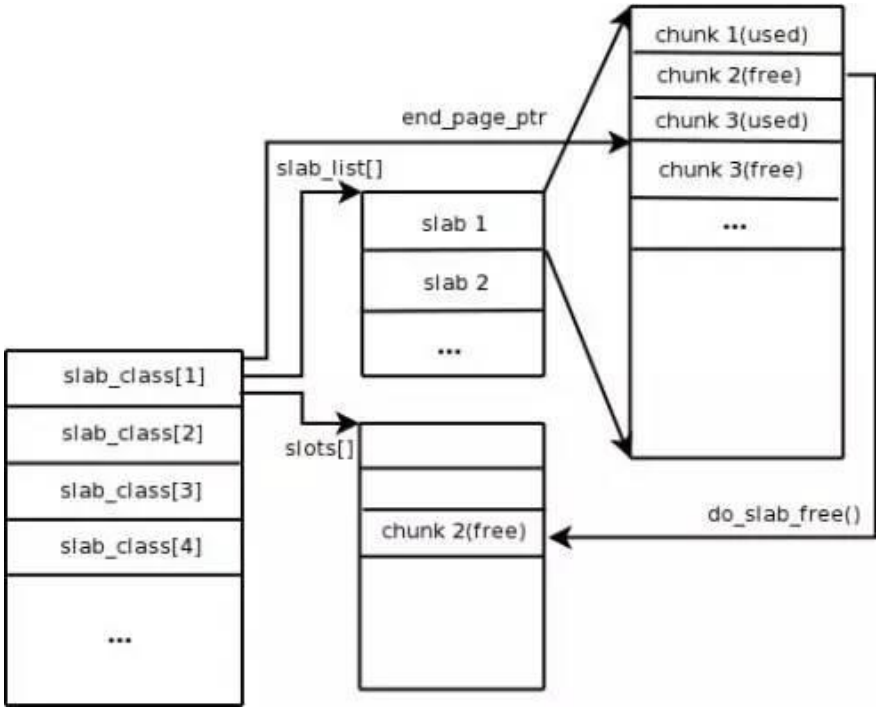
In Redis, not all data storage occurs in memory. This is a major difference between Redis and Memcached. When the physical memory is full, Redis may swap values not used for a long time to the disk. Redis only caches all the key information. If it discovers that the memory usage exceeds the threshold value, it will trigger the swap operation. Redis calculates the values for the keys to be swapped to the disk based on "swappability = age*log(size_in_memory)". It then makes these values for the keys persistent into the disk and erases them from memory. This feature enables Redis to maintain data of a size bigger than its machine memory capacity. The machine memory must keep all the keys and it will not swap all the data.

At the same time, when Redis swaps the in-memory data to the disk, the main thread that provides services, and the child thread for the swap operation will share this part of memory. So, if you update the data you intend to swap, Redis will block this operation, preventing the execution of such a change until the child thread completes the swap operation. When you read data from Redis, if the value of the read key is not in the memory, Redis needs to load the corresponding data from the swap file and then return it to the requester. Here, there is a problem of the I/O thread pool. By default, Redis will encounter congestion, that is, it will respond only after successful loading of all the swap files. This policy is suitable for batch operations when there are a small number of clients. But if you apply Redis in a large website program, it is not capable of meeting the high concurrency demands. However, you can set the I/O thread pool size for Redis running, and perform concurrent operations for reading requests for loading the corresponding data in the swap file to shorten the congestion time.

For memory-based database systems like Redis and Memcached, the memory management efficiency is a key factor influencing the system's performance. In traditional C language, malloc/free functions are the most common method for distributing and releasing memory. However, this method harbors a huge defect: first, for developers, unmatched malloc and free will easily cause memory leakage; second, the frequent calls will make it difficult to recycle and reuse a lot of memory fragments, reducing the memory utilization; and at last, the system calls will consume a far larger system overhead than the general function calls. Therefore, to improve the memory management efficiency, memory management solutions will not use the malloc/free calls directly. Both Redis and Memcached adopt their self-designed

memory management mechanisms, but the implementation methods vary a lot. Next we introduce these two mechanisms.

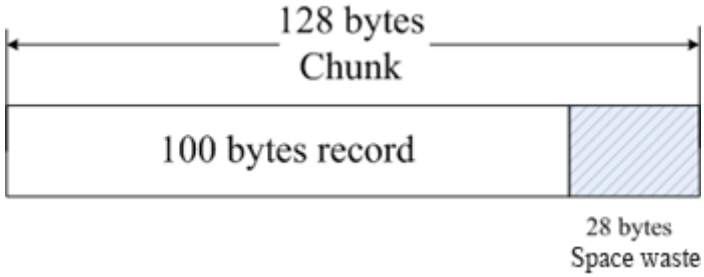
Memcached uses the Slab Allocation mechanism for memory management by default. Its main philosophy is to segment the allocated memory into chunks of a predefined specific length to store the key-value data records of the corresponding length to solve the memory fragment problem completely. Ideally, the slab's design of the allocation mechanism should guarantee external data storage, that is to say, it facilitates the storage of all the key-value data in the Slab Allocation system. However, the application of other memory requests of Memcached occur via general malloc/free calls. Normally, this is because the number and frequency of these requests determine that they will not affect the overall system performance. The principle of Slab Allocation is very simple. As shown in the figure, it first applies for a bulk of memory from the operating system and segments it into chunks of various sizes, and then groups chunks of the same size into the Slab Class. Among them, the chunk is the smallest unit for storing the key-value data. It is possible to control the size of each Slab Class by making a Growth Factor at the Memcached startup. Suppose the Growth Factor in the figure is 1.25. If the chunk in the first group is 88 bytes in size, the chunk in the second group will be 112 bytes. The remaining chunks follow the same rule.



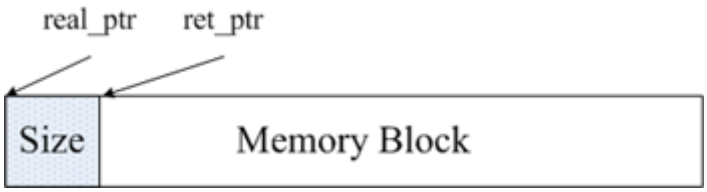
When Memcached receives the data sent from the client, it will first select the most appropriate Slab Class according to the data size, and

then query the idle chunk list containing the Slab Class in the Memcached to locate a chunk for storing the data. When a piece of data expires or is obsolete, and therefore discarded, it is possible to recycle the chunk originally occupied by the record and restore it to the idle list.

From the above process, we can see that Memcached has very high memory management efficiency that will not cause memory fragments. Its biggest defect, however, is that it may cause space waste. Because the system allocates every chunk in the memory space of a specific length, longer data might fail to utilize the space fully. As shown in the figure, when we cache data of 100 bytes into a chunk of 128 bytes, the unused 28 bytes go to waste.



Implementation of Redis' memory management mainly proceeds through the two files zmalloc.h and zmalloc.c in the source code. To facilitate memory management, Redis will store the memory size in the memory block header following memory allocation. As shown in the figure, the real_ptr is the pointer returned after Redis calls malloc. Redis stores the memory block size in the header and the memory occupied by the size is determinable, that is, the system returns the length of the size_t type, and then the ret_ptr. When the need to release memory arises, the system passes ret_ptr to the memory management program. Through the ret_ptr, the program can easily calculate the value of real_ptr and then pass real_ptr to release the memory.



Redis records the distribution of all the memory by defining an array the length of which is ZMALLOC_MAX_ALLOC_STAT. Every element in the array represents the number of memory blocks allocated by the

current program and the size of the memory block is the subscript of the element. In the source code, this array is zmalloc_allocations. The zmalloc_allocations[16] represents the number of memory blocks allocated with the length of 16 bytes. The zmalloc.c contains a static variable of used_memory to record the total size of the currently allocated memory. So in general, Redis adopts the encapsulated malloc/free, which is much simpler compared with the memory management mechanism of Memcached.

Support of data persistence

Although a memory-based store, Redis supports memory data persistence and provides two major persistence policies, RDB snapshot and AOF log. Memcached does not support data persistence operations.

RDB snapshot

Redis supports storage of the snapshot of the current data into a data file for persistence, that is, the RDB snapshot. But how can we generate the snapshot for a database with continuous data writes? Redis utilizes the copy on write mechanism of the fork command. Upon creation of a snapshot, the current process forks a subprocess that makes all the data cyclic and writes them into the RDB file. We can configure the timing of an RDB snapshot generation through the save command of Redis. For example, if you want to configure snapshot generation for once every 10 minutes, you can configure snapshot generation after each 1,000 writes. You can also configure multiple rules for implementation together. Definitions of these rules are in the configuration files of the Redis. You can also set the rules using the CONFIG SET command of Redis during the runtime of Redis without restarting Redis.

Redis' RDB file is, to an extent, incorruptible because it performs its write operations in a new process. Upon the generation of a new RDB file, the Redis-generated subprocess will first write the data into a temporary file, and then rename the temporary file into a RDB file through the atomic rename system call, so that the RDB file is always available whenever Redis suffers a fault. At the same time, the Redis' RDB file is also a link in the internal implementation of Redis' master-slave synchronization. However, RDB has its deficiency in that once the database encounters some problem, the data saved in the RDB file may be not up-to-date, and data is lost during the period from last RDB file generation to Redis failure. Note that for some businesses, this is tolerable.

AOF log

The full form of AOF log is Append Only File. It is an appended log file. Unlike the binlog of general databases, the AOF is a recognizable plaintext and its content is the Redis standard commands. Redis will only append the commands that will cause data changes to the AOF. Every command for changing data will generate a log. The AOF file will become larger and larger. Redis provides another feature—AOF rewrite. The function of AOF rewrite is to re-generate an AOF file. There is only one operation for each record in the new AOF file, instead of multiple operations for the same value recorded in the old copy. The generation process is similar to the RDB snapshot, namely forking a process, traversing the data and writing data into the new temporary AOF file. When writing data into the new file, it will write all the write operation logs to the old AOF file and record them in the memory buffering zone at the same time. Upon completing the operation, the system will write all the logs in the buffering zone to the temporary file at one time. Thereafter, it will call the atomic rename command to replace the old AOF file with the new AOF file.

AOF is a write file operation and aims to write the operation logs to the disk. It also involves the write operation procedure we mentioned earlier. After Redis calls the write operation for AOF, it uses the appendfsync option to control the time for writing the data to the disk by calling the fsync command. The three settings options in the appendfsync below have security strength from low to strong.

- appendfsync no: when we set the appendfsync to no, Redis will not take the initiative to call fsync to synchronize the AOF logs to the disk. The synchronization will be fully dependent on the operating system debugging. Most Linux operating systems perform the fsync operation once every 30 seconds to write the data in the buffering zone to the disk.

- appendfsync everysec: when we set appendfsync to everysec, Redis will call fsync once every other second by default to write data in the buffering zone to the disk. But when an fsync call lasts for more than 1 second, Redis will adopt fsync delay to wait for another second. That is, Redis will call the fsync after two seconds. It will perform this fsync no matter how long it takes to execute. At this time, because the file descriptor will experience congestion during the file fsync, the current write operation will experience similar congestion. The conclusion is that in a vast majority of cases, Redis will perform fsync once every other second. In the worst cases, it will perform the fsync operation once every two

seconds. Most database systems refer to this operation as group commit, that is, combing the data of multiple writes and write the logs to the disk at a time.

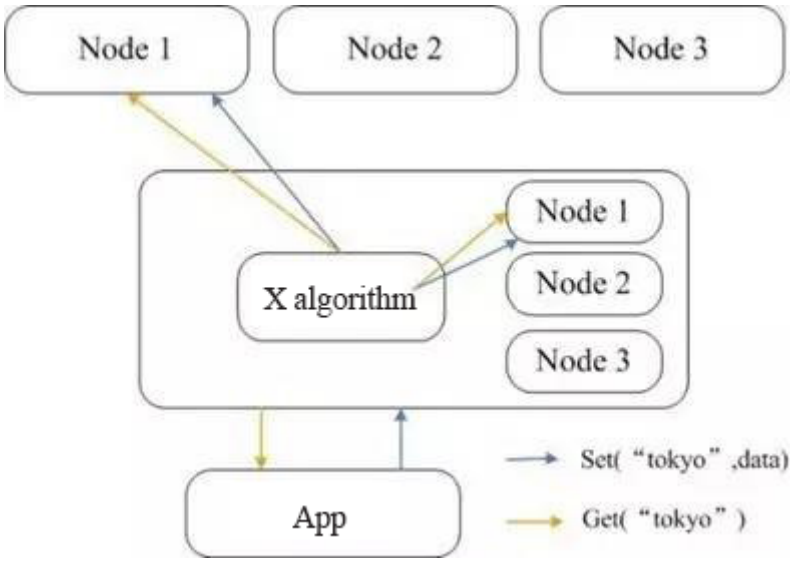
- appednfsync always: when we set appendfsync to always, every write operation will call fsync once. At this time, the data is the most secure. Of course, since it performs fsync every time, it will compromise the performance.

For general business requirements, we suggest you use RDB for persistence because the RDB overhead is much lower than that of AOF logs. For applications that cannot stand the risk of any data loss, we recommend you use AOF logs.

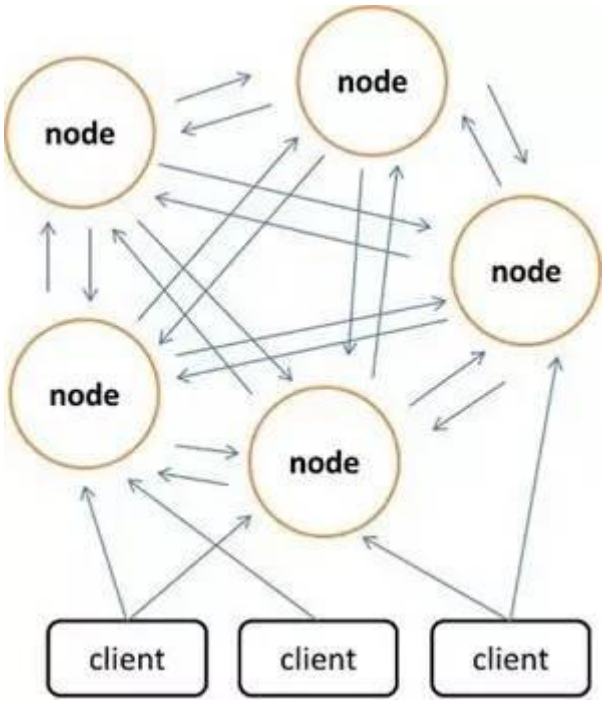
Difference in Cluster Management

Memcached is a full-memory data buffering system. Although Redis supports data persistence, the full-memory is the essence of its high performance. For a memory-based store, the size of the memory of the physical machine is the maximum data storing capacity of the system. If the data size you want to handle surpasses the physical memory size of a single machine, you need to build distributed clusters to expand the storage capacity.

Memcached itself does not support distributed mode. You can only achieve the distributed storage of Memcached on the client side through distributed algorithms such as Consistent Hash. The figure below demonstrates the distributed storage implementation schema of Memcached. Before the client side sends data to the Memcached cluster, it first calculates the target node of the data through the nested distributed algorithm that in turn directly sends the data to the node for storage. But when the client side queries the data, it also needs to calculate the node that serves as the location of the data queried, and then send the query request to the node directly to get the data.



Compared with Memcached which can only achieve distributed storage on the client side, Redis prefers to build distributed storage on the server side. The latest version of Redis supports distributed storage. Redis Cluster is an advanced version of Redis that achieves distributed storage and allows SPOF. It has no central node and is capable of linear expansion. The figure below provides the distributed storage architecture of Redis Cluster. The inter-node communication follows the binary protocol but the node-client communication follows the ASCII protocol. In the data placement policy, Redis Cluster divides the entire key numerical range into 4,096 hash slots and allows the storage of one or more hash slots on each node. That is to say, the current Redis Cluster supports a maximum of 4,096 nodes. The distributed algorithm that Redis Cluster uses is also simple: crc16 (key) % HASH_SLOTS_NUMBER.



Redis Cluster introduces the master node and slave node to ensure data availability in case of SPOF. Every master node in Redis Cluster has two corresponding slave nodes for redundancy. As a result, any two failed nodes in the whole cluster will not impair data availability. When the master node exists, the cluster will automatically choose a slave node to become the new master node.

Conclusion

In this article, we have discussed the differences between Redis and Memcached. We started by listing few points of comparison suggested by Salvatore Sanfilippo, the author of Redis. Thereafter, we further elaborated, with key points of difference between Redis and Memcached being the data-types supported, cluster management, data persistence support, and memory management schemes.

To learn more, visit the Alibaba Cloud ApsaraDB for Redis page and the ApsaraDB for Memcache page.

Reference:

https://www.alibabacloud.com/blog/redis-vs-memcached-in-memory-data-storage-systems_592091?spm=a2c41.11544560.0.0